

Padrões e Arquitetura de Software

EFC 1: Refatoração guiada por SOLID, Clean Code e padrões GoF

Sistema de pedidos da Loja Verde

Felipe Cosmo Granziol (RA 24021602)

Guilherme Bars (RA 24014122)

Gustavo Kurten (RA 24008150)

Henrique Gambin (RA 24013762)

João Celso (RA 24012463)

Pedro Tiezo Sales Shimizu (RA 24005158)

Vitor Hugo Barbosa (RA 24018852)

Faculdade de Análise de Sistemas, PUC-Campinas, 2026

Documento de análise: EFC1 Loja Verde

As referências de linha apontam para `legacy.py`, o código original antes da refatoração.

(a) Identificação das violações SOLID

SRP (responsabilidade única)

O método `Sis.add_ped` (linhas 15 a 49) faz quatro coisas ao mesmo tempo: calcula o desconto de cada item, aplica o desconto do cliente, grava o pedido no banco e dispara as notificações. São quatro motivos diferentes para esse método mudar. Se a regra de notificação mudar, o cálculo do total fica exposto ao risco de quebrar junto.

O método `Sis.upd_st` (linhas 59 a 80) tem o mesmo problema. Ele atualiza o estado do pedido, manda e-mail e SMS e ainda calcula os pontos de fidelidade. A classe `Sis` inteira concentra pedido, pagamento, estoque, relatório e notificação, com mais de sete métodos públicos.

OCP (aberto/fechado)

`Sis.proc_pag` (linhas 116 a 139) decide o pagamento com uma cadeia de `if/elif` sobre `cartao`, `pix` e `boleto`. Para aceitar criptomoeda, é preciso editar o método. `Sis.add_ped` (linhas 18 a 31) repete o padrão para o tipo de desconto do item e o tipo de cliente. Qualquer desconto novo obriga a mexer na classe que já funciona. As três extensões pedidas no trabalho seriam impossíveis sem alterar código existente.

LSP (substituição de Liskov)

`PedEspecial.upd_st` (linhas 183 a 190) pula direto para qualquer estado e não envia notificação nem concede pontos. Isso contraria o comportamento observável de `Sis.upd_st`. `PedEspecial.add_ped` (linhas 164 a 181) reescreve o cálculo, acrescenta 15% e troca a notificação. Um trecho de código que espera um `Sis` recebe um `PedEspecial` e passa a ter outro resultado sem perceber.

ISP (segregação de interfaces)

Quem só precisa emitir um relatório depende, através de `Sis`, também de pagamento, estoque e notificação. Não há interfaces separadas por colaboração. A subclasse `PedEspecial` é forçada a herdar toda a superfície de `Sis` para usar apenas parte dela, reimplementando o resto.

DIP (inversão de dependência)

A regra de negócio fala SQL direto com `sqlite3` (linha 8). Trocar a persistência significa reescrever a classe. As notificações saem por `print`, e o estoque vem de um `dict` fixo com um `# TODO: integrar com sistema externo` (linha 143). O código de alto nível depende de detalhes concretos, o que impede testar em isolamento ou trocar o canal de saída.

(b) Soluções implementadas

Violação	Solução	Princípio	Padrão GoF
add_ped/upd_st faziam tudo	Separação em OrderService, PaymentService, PricingService e ReportService	SRP	
if/elif de pagamento	PaymentStrategy com dicionário injetado	OCP	Strategy
if/elif de desconto	ItemPricingRule e CustomerDiscountStrategy	OCP	Strategy
Notificação acoplada à regra	OrderNotifier com NotificationChannel	SRP, OCP	Observer
Criação por tipo de cliente	OrderFactory e suas subclasses	OCP	Factory Method
sqlite3 direto na regra	OrderRepository (ABC) com SqliteOrderRepository	DIP	Repository
print e estoque fixo	MessageSink, StockService e Clock como abstrações	DIP	
Herança PedEspecial	Eliminada; o acréscimo vira regra de preço, mesmo tipo Order	LSP	

O caso do pagamento ilustra bem a mudança. Antes:

```
if m == 'cartao': ...; self.upd_st(id, 'aprovado'); return True
elif m == 'pix': ...; self.upd_st(id, 'aprovado'); return True
elif m == 'boleto': ...; return True
```

Depois, em payment_service.py e strategies/payment_strategy.py:

```
strategy = self._strategies.get(method)          # injetado no construtor
result = strategy.charge(order)
if result.auto_approve:
    self._order_service.advance_status(order.id, OrderStatus.APROVADO)
```

Para aceitar criptomoeda basta uma classe nova, CryptoPayment, e uma linha que a registra no composition root. Nenhuma classe antiga muda.

(c) Diagrama UML de classes

O arquivo docs/diagrama.puml tem a fonte em PlantUML, e docs/loja_verde.png mostra o resultado renderizado. As camadas aparecem explícitas. As setas de dependência apontam para abstrações, conforme o DIP: os serviços dependem de OrderRepository, PaymentStrategy e NotificationChannel, nunca das classes concretas.

(d) Melhorias de Clean Code

Os nomes passaram a dizer o que fazem. `Sis` virou `OrderService` e `SqliteOrderRepository`. `add_ped` virou `create_order`, `proc_pag` virou `process` e `upd_st` virou `advance_status`. Os parâmetros `n`, `its` e `t` viraram `customer`, `items` e o tipo do cliente.

Os números mágicos saíram. As multiplicações soltas (`* 0.95`, `* 0.9`, `* 1.15`, `* 2`) viraram estratégias e constantes com nome, como `VipDiscount`, `CRYPTO_FEE_RATE` e `VOLUME_DISCOUNT_FACTOR`. Os métodos ficaram curtos, abaixo de vinte linhas, com uma responsabilidade cada.

A tipagem ficou forte. `CustomerType` e `OrderStatus` são Enum no lugar de strings soltas, as entidades são `dataclass` e o `mypy --strict` roda sem erro. O cálculo de total, que aparecia duplicado entre `Sis` e `PedEspecial`, ficou em um lugar só, no `PricingService`.

(e) Extensões implementadas

As três features são a prova de que o OCP foi cumprido. Dá para auditar pelo `git log` e pelo `git diff`: cada uma é um commit que só adiciona arquivos em `src/extensions/` e nos testes, mais a fiação no `composition root` (`src/main.py`). Nenhuma classe de domínio foi tocada.

A criptomoeda com taxa de 2% está em `extensions/crypto_payment.py`, como `CryptoPayment(PaymentStrategy)` registrada no dicionário de estratégias. O WhatsApp para todos os clientes está em `extensions/whatsapp_channel.py`, como `WhatsAppChannel(NotificationChannel)` inscrito como canal global. O desconto por volume de 15% para três ou mais unidades está em `extensions/volume_discount.py`, como `VolumeDiscountRule(ItemPricingRule)` anexada à cadeia de regras de preço.

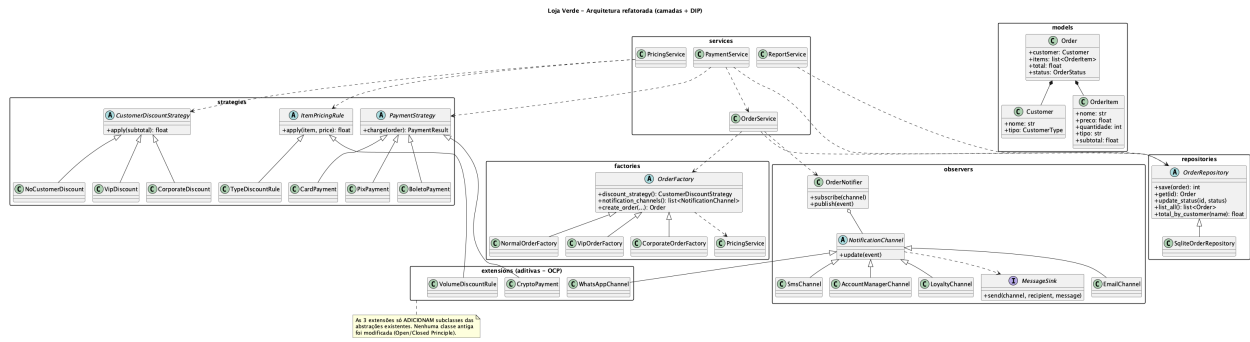
Ficou simples porque as abstrações certas já existiam e são injetadas: estratégia de pagamento, canal de notificação e regra de precificação. A extensão se resume a plugar uma subclasse nova.

(f) Reflexão metacognitiva

O princípio que eu ainda não domino é o LSP. A violação clássica, a do `PedEspecial`, eu reconheço de cara. O que me trava são os casos sutis: quando uma subclasse aperta a pré-condição em vez de só estender o comportamento. Sem escrever um teste de substituição explícito, ainda não consigo afirmar com segurança se uma hierarquia respeita Liskov.

Usei IA para acelerar a montagem das camadas e a redação dos testes de caracterização. O que funcionou bem foi pedir o mapeamento de cada violação para o padrão correspondente. Discordei em um ponto: a primeira sugestão colocava a escolha dos canais de notificação dentro do `OrderService`, com um `if` por tipo de cliente. Isso recriaria a violação de OCP que eu estava tentando consertar. Movi essa decisão para as fábricas (`OrderFactory`), de modo que o serviço fica fechado para modificação.

Anexo: diagrama UML renderizado



Fonte em PlantUML: docs/diagrama.puml. Imagem em alta resolução: docs/loja_verde.png (amplie no leitor de PDF para ler os detalhes).

Saída dos testes (pytest -v)

```
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/guimac/Documents/trabalhodoug1
collecting ... collected 65 items
extensions/test_crypto_payment.py::test_taxa_de_2_por_cento PASSED [ 1%]
extensions/test_crypto_payment.py::test_cripto_aprova_pedido_no_fluxo_completo PASSED [ 3%]
extensions/test_volume_discount.py::test_aplica_15_por_cento_a_partir_de_3_unidades PASSED [ 4%]
extensions/test_volume_discount.py::test_nao_aplica_abaixo_de_3_unidades PASSED [ 6%]
extensions/test_volume_discount.py::test_fluxo_completo_com_desconto_de_volume PASSED [ 7%]
extensions/test_volume_discount.py::test_sem_extensao_nao_ha_desconto_de_volume PASSED [ 9%]
extensions/test_whatsapp_channel.py::test_whatsapp_notifica_cliente_normal PASSED [ 10%]
extensions/test_whatsapp_channel.py::test_whatsapp_disponivel_para_todos_os_tipos PASSED [ 12%]
extensions/test_whatsapp_channel.py::test_whatsapp_ausente_sem_extensoes PASSED [ 13%]
golden_master/test_equivalence.py::test_total_identico_ao_legado[normal-CustomerType.NORMAL-itens_ra PASSED [ 14%]
golden_master/test_equivalence.py::test_total_identico_ao_legado[vip-CustomerType.VIP-itens_raw1] PASSED [ 15%]
golden_master/test_equivalence.py::test_total_identico_ao_legado[corporativo-CustomerType.CORPORATIV PASSED [ 16%]
golden_master/test_legacy_behavior.py::test_pedido_normal_calcula_total_corretamente PASSED [ 20%]
golden_master/test_legacy_behavior.py::test_pedido_vip_aplica_desconto_de_5_por_cento PASSED [ 21%]
golden_master/test_legacy_behavior.py::test_pedido_corporativo_aplica_desconto_de_10_por_cento PASSED [ 22%]
golden_master/test_legacy_behavior.py::test_desc20_no_item PASSED [ 24%]
golden_master/test_legacy_behavior.py::test_pagamento_insuficiente_falha PASSED [ 26%]
golden_master/test_legacy_behavior.py::test_pix_aprova_pedido_automaticamente PASSED [ 27%]
golden_master/test_legacy_behavior.py::test_cartao_aprova_pedido_automaticamente PASSED [ 29%]
golden_master/test_legacy_behavior.py::test_boleto_nao_aprova_automaticamente PASSED [ 30%]
golden_master/test_legacy_behavior.py::test_atualizacao_de_status PASSED [ 32%]
golden_master/test_legacy_behavior.py::test_cancelamento PASSED [ 33%]
golden_master/test_legacy_behavior.py::test_geracao_de_relatorios PASSED [ 35%]
golden_master/test_legacy_pin.py::test_pedido_normal_calcula_total PASSED [ 36%]
golden_master/test_legacy_pin.py::test_pedido_vip_desconto_5 PASSED [ 38%]
golden_master/test_legacy_pin.py::test_pagamento_insuficiente_falha PASSED [ 40%]
golden_master/test_legacy_pin.py::test_pix_aprova_automaticamente PASSED [ 41%]
golden_master/test_legacy_pin.py::test_boleto_nao_aprova PASSED [ 43%]
integration/test_main_smoke.py::test_main_executa_completo PASSED [ 44%]
integration/test_order_service.py::test_criacao_vip_dispara_email_e_sms PASSED [ 46%]
integration/test_order_service.py::test_criacao_corporativa_notifica_gerente PASSED [ 47%]
integration/test_order_service.py::test_entrega_concede_pontos_de_fidelidade PASSED [ 49%]
integration/test_order_service.py::test_advance_status_pedido_inexistente PASSED [ 50%]
integration/test_order_service.py::test_cancelar_pedido_inexistente PASSED [ 52%]
integration/test_payment_service.py::test_metodo_invalido_retorna_false PASSED [ 53%]
integration/test_payment_service.py::test_pagamento_pedido_inexistente PASSED [ 55%]
integration/test_payment_service.py::test_valor_insuficiente_avisa PASSED [ 56%]
integration/test_report_service.py::test_relatorio_de_vendas PASSED [ 58%]
integration/test_report_service.py::test_relatorio_de_clientes_sem_duplicar PASSED [ 60%]
integration/test_repository.py::test_save_e_get_roundtrip PASSED [ 61%]
integration/test_repository.py::test_get_inexistente_retorna_none PASSED [ 63%]
integration/test_repository.py::test_update_status PASSED [ 64%]
integration/test_repository.py::test_total_by_customer PASSED [ 66%]
integration/test_repository.py::test_list_all PASSED [ 67%]
unit/test_discount_strategy.py::test_type_discount_rule[normal-200.0] PASSED [ 69%]
unit/test_discount_strategy.py::test_type_discount_rule[desc10-180.0] PASSED [ 70%]
unit/test_discount_strategy.py::test_type_discount_rule[desc20-160.0] PASSED [ 72%]
unit/test_discount_strategy.py::test_type_discount_rule[frete_gratis-200.0] PASSED [ 73%]
unit/test_discount_strategy.py::test_type_discount_rule[desconhecido-200.0] PASSED [ 75%]
unit/test_discount_strategy.py::test_no_customer_discount PASSED [ 76%]
unit/test_discount_strategy.py::test_vip_discount PASSED [ 78%]
unit/test_discount_strategy.py::test_corporate_discount PASSED [ 80%]
unit/test_notification.py::test_email_channel_reage_a_criacao PASSED [ 81%]
unit/test_notification.py::test_sms_ignora_eventos_de_entrega PASSED [ 83%]
unit/test_notification.py::test_loyalty_concede_pontos_na_entrega PASSED [ 84%]
unit/test_notification.py::test_loyalty_so_dispara_na_entrega PASSED [ 86%]
unit/test_order_factory.py::test_normal_factory_usa_desconto_neutro PASSED [ 87%]
unit/test_order_factory.py::test_vip_factory_inclui_sms PASSED [ 89%]
unit/test_order_factory.py::test_corporate_factory_inclui_gerente PASSED [ 90%]
unit/test_payment_strategy.py::test_cartao_aprova_automaticamente PASSED [ 92%]
unit/test_payment_strategy.py::test_pix_aprova_automaticamente PASSED [ 93%]
unit/test_payment_strategy.py::test_boleto_nao_aprova_automaticamente PASSED [ 95%]
```

```
unit/test_services_misc.py::test_system_clock_format PASSED [ 96%]  
unit/test_services_misc.py::test_estoque_indisponivel_para_item_inexistente PASSED [ 98%]  
unit/test_services_misc.py::test_order_service_sem_fabrica_para_o_tipo PASSED [100%]  
===== 65 passed in 0.08s =====
```

Relatório de métricas

Métrica	Ferramenta	Mínimo exigido	Obtido
Cobertura de testes	coverage.py	≥ 85%	98%
Lint / PEP 8	ruff	sem erros	0 erros
Complexidade ciclomática	radon cc	nota máxima B	média A
Type hints	mypy --strict	sem erros	0 erros
Testes	pytest	todos passando	65 passando

Cobertura por módulo (coverage.py):

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/__init__.py	0	0	0	0	100%	
src/extensions/__init__.py	0	0	0	0	100%	
src/extensions/crypto_payment.py	9	0	0	0	100%	
src/extensions/volume_discount.py	10	0	2	0	100%	
src/extensions/whatsapp_channel.py	8	0	2	1	90%	28->exit
src/factories/__init__.py	0	0	0	0	100%	
src/factories/order_factory.py	35	0	0	0	100%	
src/main.py	69	0	8	3	96%	115->125, 126->13
src/models/__init__.py	0	0	0	0	100%	
src/models/customer.py	11	0	0	0	100%	
src/models/order.py	19	0	0	0	100%	
src/models/order_item.py	11	0	0	0	100%	
src/observers/__init__.py	0	0	0	0	100%	
src/observers/notification_observer.py	58	1	10	1	97%	41, 89->exit
src/repositories/__init__.py	0	0	0	0	100%	
src/repositories/interfaces.py	14	0	0	0	100%	
src/repositories/sqlite_order_repository.py	41	0	0	0	100%	
src/services/__init__.py	0	0	0	0	100%	
src/services/clock.py	9	1	0	0	89%	12
src/services/order_service.py	53	0	12	1	98%	63->65
src/services/payment_service.py	29	0	8	0	100%	
src/services/pricing_service.py	17	0	4	0	100%	
src/services/report_service.py	27	0	8	0	100%	
src/services/stock_service.py	12	0	0	0	100%	
src/strategies/__init__.py	0	0	0	0	100%	
src/strategies/discount_strategy.py	23	0	0	0	100%	
src/strategies/payment_strategy.py	22	0	0	0	100%	
TOTAL	477	2	54	6	98%	

Lint, tipagem e complexidade:

```
$ ruff check .
All checks passed!

$ mypy --strict src/
Success: no issues found in 27 source files

$ radon cc src/ -s -a
119 blocks (classes, functions, methods) analyzed.
Average complexity: A (1.7394957983193278)
```